```
/* ============================================================
 *  STM32F429VIT6 — Combined 4-Wheel Robot + 6-DOF Arm Controller
 * ============================================================
 *
 *  MODE SWITCHING:
 *    Send 'M' to toggle between Robot mode and Arm mode.
 *    On startup the board is in ROBOT mode.
 *
 * ── ROBOT MODE (4-wheel drive via TB6612FNG) ──────────────────
 *  Commands (ASCII bytes from ESP32 app):
 *    F   Move forward
 *    B   Move backward
 *    L   Turn left
 *    R   Turn right
 *    S   Stop
 *
 *  Motor PWM:  TIM1  (PA8=CH1, PA9=CH2, PA10=CH3, PA11=CH4)
 *  Direction GPIOs:
 *    Motor 4:  PD4 / PD5
 *    Motor 3:  PD2 / PD3
 *    Motor 1A: PC4 / PC5
 *    Motor 1B: PD0 / PD1
 *  STBY:       PC0  (HIGH = drivers enabled)
 *
 * ── ARM MODE (6-DOF servo arm via Hiwonder 500–2500 µs) ──────
 *  Commands (numeric bytes from ESP32 app):
 *    1/2    Base rotate right/left
 *    3/4    Shoulder up/down
 *    5/6    Elbow up/down
 *    7/8    Wrist rotate CW/CCW
 *    9/10   Wrist pitch up/down
 *    11/12  Gripper open/close
 *    13     Save current position
 *    14     Reset / stop sequence
 *    15     Run saved sequence
 *    16     Pause (send 15 to resume, 14 to stop)
 *    H      Home all servos to 1500 µs
 *    101–249  Set servo speed (value / 10 = ms per step)
 *
 *  Servo GPIO / Timer mapping:
 *
 *  ┌──────────┬──────┬──────────┬──────────────────────────┐
 *  │ Servo    │ GPIO │ Timer    │ PCB connector            │
 *  ├──────────┼──────┼──────────┼──────────────────────────┤
 *  │ Base     │ PD12 │ TIM4_CH1 │ LS2 pad  pin1=SIGNAL     │
```

```
 *  │ Shoulder   │ PD13  │ TIM4_CH2  │ LS1 pad  pin1=SIGNAL  │
 *  │ Elbow      │ PA1   │ TIM2_CH2  │ PH1 hdr  pin1=SIGNAL  │
 *  │ Wrist R    │ PD14  │ TIM4_CH3  │ JP8 pad  pin1=SIGNAL  │
 *  │ Wrist P    │ PB15  │ TIM12_CH2 │ LS3 pad  pin1=SIGNAL  │
 *  │ Gripper    │ PA2   │ TIM2_CH3  │ PH2 hdr  pin1=SIGNAL  │
 *  └────────────┴───────┴───────────┴──────────────────────┘
 *
 *  BLUETOOTH: UART7 (PE7=RX, PE8=TX) — 115200 baud
 * ========================================================== */

#include "main.h"
#include <stdlib.h>
#include <string.h>
#include <stdio.h>

/* ── Timer / UART handles ──────────────────────────────────────────── */
TIM_HandleTypeDef  htim1;   /* Robot motor PWM  — TIM1 CH1-CH4              */
TIM_HandleTypeDef  htim2;   /* Servo 50 Hz      — PA1(CH2) PA2(CH3)         */
TIM_HandleTypeDef  htim4;   /* Servo 50 Hz      — PD12(CH1) PD13(CH2) PD14(CH3) */
TIM_HandleTypeDef  htim12;  /* Servo 50 Hz      — PB15(CH2)                 */
UART_HandleTypeDef huart7;  /* Bluetooth 115200 baud                        */

/* ── Robot defines ─────────────────────────────────────────────────────*/
#define MOTOR_SPEED  750    /* 75 % duty  (max = MOTOR_MAX) */
#define MOTOR_MAX   1000    /* matches TIM1 Period = 999     */

/* ── Arm defines ──────────────────────────────────────────────────────*/
#define SERVO_MIN_US    500
#define SERVO_MAX_US   2500
#define SERVO_MID_US   1500
#define NUM_SERVOS        6
#define MAX_STEPS        50
#define SERVO_STEP_SIZE  10

#define S_BASE      0   /* PD12  TIM4_CH1   LS2 pad */
#define S_SHOULDER  1   /* PD13  TIM4_CH2   LS1 pad */
#define S_ELBOW     2   /* PA1   TIM2_CH2   PH1 hdr */
#define S_WRIST_R   3   /* PD14  TIM4_CH3   JP8 pad */
#define S_WRIST_P   4   /* PB15  TIM12_CH2  LS3 pad */
#define S_GRIPPER   5   /* PA2   TIM2_CH3   PH2 hdr */

/* ── Global state ──────────────────────────────────────────────────────*/
typedef enum { MODE_ROBOT = 0, MODE_ARM = 1 } ControlMode;
ControlMode currentMode = MODE_ROBOT;
```

```
/* Arm globals */
int32_t servoPos[NUM_SERVOS] = {
  SERVO_MID_US, SERVO_MID_US, SERVO_MID_US,
  SERVO_MID_US, SERVO_MID_US, SERVO_MID_US
};
int32_t savedSteps[MAX_STEPS][NUM_SERVOS];
int     stepCount       = 0;
int     speedDelay      = 20;
uint8_t servoPwmStarted = 0;

/* ── Prototypes ──────────────────────────────────────────────────────────*/
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_TIM1_Init(void);
static void MX_TIM2_Init(void);
static void MX_TIM4_Init(void);
static void MX_TIM12_Init(void);
static void MX_UART7_Init(void);

/* Robot */
void Move_Robot(int16_t m4, int16_t m3, int16_t m1a, int16_t m1b);
void Stop_Robot(void);

/* Arm */
void Servo_EnablePWM(void);
void Servo_WritePulse(uint8_t id, int32_t pulseUs);
void Servo_Step(uint8_t id, int8_t direction);
void Servo_Home(void);
void Run_Sequence(void);

/* Shared */
void UART_Print(const char *msg);
void Print_Mode(void);

/* ============================================================
 *  MAIN
 * ============================================================ */
int main(void)
{
  HAL_Init();
  SystemClock_Config();

  MX_GPIO_Init();
```

```
MX_TIM1_Init();
MX_TIM2_Init();
MX_TIM4_Init();
MX_TIM12_Init();
MX_UART7_Init();

/* Start robot motor PWM channels */
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_1);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_3);
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_4);

/* Enable TB6612FNG drivers: STBY = PC0 HIGH */
HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0, GPIO_PIN_SET);
HAL_Delay(100);

Stop_Robot();

UART_Print("=== MecaArm Controller Ready ===\r\n");
UART_Print("Wheels: F/B/L/R/S | Arm: numeric bytes 1-15 | 0=arm stop | H=home\r\n");

uint8_t rx   = 0;
uint8_t armM = 0;
char dbg[64];

while (1)
{
  if (HAL_UART_Receive(&huart7, &rx, 1, 10) == HAL_OK)
  {

    sprintf(dbg, "RX: %c (%d)\r\n",
          (rx >= 32 && rx < 127) ? rx : '.', rx);
    UART_Print(dbg);

    HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);

    /* ======================================================
     * ARM STOP: app sends byte 0 on button release
     * ====================================================== */
    if (rx == 0)
    {
      armM = 0;
      goto loop_end;
    }
```

```
/* ======================================================
 * WHEEL COMMANDS: ASCII bytes from app
 * ====================================================== */
if (rx == 'F')
{
  Move_Robot(MOTOR_SPEED, MOTOR_SPEED, MOTOR_SPEED, MOTOR_SPEED);
  armM = 0;
  UART_Print("MOVE FORWARD\r\n");
  goto loop_end;
}

if (rx == 'B')
{
  Move_Robot(-MOTOR_SPEED, -MOTOR_SPEED, -MOTOR_SPEED, -MOTOR_SPEED);
  armM = 0;
  UART_Print("MOVE BACKWARD\r\n");
  goto loop_end;
}

if (rx == 'R')
{
  Move_Robot(-MOTOR_SPEED, -MOTOR_SPEED, MOTOR_SPEED, MOTOR_SPEED);
  armM = 0;
  UART_Print("TURN RIGHT\r\n");
  goto loop_end;
}

if (rx == 'L')
{
  Move_Robot(MOTOR_SPEED, MOTOR_SPEED, -MOTOR_SPEED, -MOTOR_SPEED);
  armM = 0;
  UART_Print("TURN LEFT\r\n");
  goto loop_end;
}

if (rx == 'S')
{
  Stop_Robot();
  armM = 0;
  UART_Print("STOP\r\n");
  goto loop_end;
}
```

```
/* ======================================================
 * ARM HOME: ASCII H
 * ====================================================== */
if (rx == 'H')
{
  Servo_EnablePWM();
  Servo_Home();
  armM = 0;
  UART_Print("Servos homed.\r\n");
  goto loop_end;
}

/* ======================================================
 * SAVE POSITION: raw byte 13
 * ====================================================== */
if (rx == 13)
{
  Servo_EnablePWM();

  if (stepCount < MAX_STEPS)
  {
    for (int i = 0; i < NUM_SERVOS; i++)
      savedSteps[stepCount][i] = servoPos[i];

    stepCount++;
    UART_Print("Position saved.\r\n");
  }
  else
  {
    UART_Print("Sequence full.\r\n");
  }

  armM = 0;
  goto loop_end;
}

/* ======================================================
 * RESET SEQUENCE: raw byte 14
 * ====================================================== */
if (rx == 14)
{
  stepCount = 0;
  armM = 0;
  Stop_Robot();
```

```
  UART_Print("Sequence reset.\r\n");
  goto loop_end;
}

/* ======================================================
 * RUN SEQUENCE: raw byte 15
 * ====================================================== */
if (rx == 15)
{
  Servo_EnablePWM();
  Run_Sequence();
  armM = 0;
  goto loop_end;
}

/* ======================================================
 * PAUSE: raw byte 16 or 18 depending on app
 * Your app seems to send 18, so accept both.
 * ====================================================== */
if (rx == 16 || rx == 18)
{
  armM = 0;
  UART_Print("Pause command received.\r\n");
  goto loop_end;
}

/* ======================================================
 * SPEED: raw bytes 101-249
 * ====================================================== */
if (rx >= 101 && rx <= 249)
{
  speedDelay = rx / 10;
  sprintf(dbg, "Speed set: %d ms/step\r\n", speedDelay);
  UART_Print(dbg);
  armM = 0;
  goto loop_end;
}

/* ======================================================
 * ARM MOVEMENT: raw numeric bytes 1-12
 * ====================================================== */
if (rx >= 1 && rx <= 12)
{
  armM = rx;
```

```
      Servo_EnablePWM();
      goto loop_end;
    }
  }

loop_end:

  /* Continuous arm movement while app button is held */
  switch (armM)
  {
    case 1:
      Servo_Step(S_BASE, +1);
      HAL_Delay(speedDelay);
      break;

    case 2:
      Servo_Step(S_BASE, -1);
      HAL_Delay(speedDelay);
      break;

    case 3:
      Servo_Step(S_SHOULDER, +1);
      HAL_Delay(speedDelay);
      break;

    case 4:
      Servo_Step(S_SHOULDER, -1);
      HAL_Delay(speedDelay);
      break;

    case 5:
      Servo_Step(S_ELBOW, +1);
      HAL_Delay(speedDelay);
      break;

    case 6:
      Servo_Step(S_ELBOW, -1);
      HAL_Delay(speedDelay);
      break;

    case 7:
      Servo_Step(S_WRIST_R, +1);
      HAL_Delay(speedDelay);
      break;
```

```
      case 8:
        Servo_Step(S_WRIST_R, -1);
        HAL_Delay(speedDelay);
        break;

      case 9:
        Servo_Step(S_WRIST_P, +1);
        HAL_Delay(speedDelay);
        break;

      case 10:
        Servo_Step(S_WRIST_P, -1);
        HAL_Delay(speedDelay);
        break;

      case 11:
        Servo_Step(S_GRIPPER, +1);
        HAL_Delay(speedDelay);
        break;

      case 12:
        Servo_Step(S_GRIPPER, -1);
        HAL_Delay(speedDelay);
        break;

      default:
        break;
    }
  }
}

/* ============================================================
 *  ROBOT MOTOR CONTROL
 * ============================================================ */
void Move_Robot(int16_t m4, int16_t m3, int16_t m1a, int16_t m1b)
{
  /* Motor 4 — PD4/PD5 direction, TIM1_CH1 speed */
  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_4, (m4 >= 0) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_5, (m4 >= 0) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_1, (uint32_t)abs(m4));
```

```
  /* Motor 3 — PD2/PD3 direction, TIM1_CH2 speed */
  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_2, (m3 >= 0) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_3, (m3 >= 0) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_2, (uint32_t)abs(m3));

  /* Motor 1A — PC4/PC5 direction, TIM1_CH3 speed */
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_4, (m1a >= 0) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_5, (m1a >= 0) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_3, (uint32_t)abs(m1a));

  /* Motor 1B — PD0/PD1 direction, TIM1_CH4 speed */
  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_0, (m1b >= 0) ? GPIO_PIN_SET :
GPIO_PIN_RESET);
  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_1, (m1b >= 0) ? GPIO_PIN_RESET :
GPIO_PIN_SET);
  __HAL_TIM_SET_COMPARE(&htim1, TIM_CHANNEL_4, (uint32_t)abs(m1b));
}

void Stop_Robot(void)
{
  Move_Robot(0, 0, 0, 0);
}

/* ============================================================
 *  SERVO / ARM CONTROL
 * ============================================================ */
void Servo_EnablePWM(void)
{
  if (servoPwmStarted) return;

  HAL_TIM_PWM_Start(&htim2,  TIM_CHANNEL_2);  /* PA1  Elbow    */
  HAL_TIM_PWM_Start(&htim2,  TIM_CHANNEL_3);  /* PA2  Gripper  */
  HAL_TIM_PWM_Start(&htim4,  TIM_CHANNEL_1);  /* PD12 Base     */
  HAL_TIM_PWM_Start(&htim4,  TIM_CHANNEL_2);  /* PD13 Shoulder */
  HAL_TIM_PWM_Start(&htim4,  TIM_CHANNEL_3);  /* PD14 Wrist R  */
  HAL_TIM_PWM_Start(&htim12, TIM_CHANNEL_2);  /* PB15 Wrist P  */

  servoPwmStarted = 1;
}
```

```
/**
 * @brief Write pulse width in microseconds to one servo.
 *        Timer clock = 1 MHz (PSC=15 at 16 MHz HSI)
 *        → 1 tick = 1 µs → CCR = pulseUs directly.
 *        Clamped to Hiwonder range 500–2500 µs.
 */
void Servo_WritePulse(uint8_t id, int32_t pulseUs)
{
  if (id >= NUM_SERVOS) return;
  Servo_EnablePWM();

  if (pulseUs < SERVO_MIN_US) pulseUs = SERVO_MIN_US;
  if (pulseUs > SERVO_MAX_US) pulseUs = SERVO_MAX_US;
  servoPos[id] = pulseUs;

  switch (id)
  {
    case S_BASE:     __HAL_TIM_SET_COMPARE(&htim4,  TIM_CHANNEL_1,
(uint32_t)pulseUs); break;
    case S_SHOULDER: __HAL_TIM_SET_COMPARE(&htim4,  TIM_CHANNEL_2,
(uint32_t)pulseUs); break;
    case S_ELBOW:    __HAL_TIM_SET_COMPARE(&htim2,  TIM_CHANNEL_2,
(uint32_t)pulseUs); break;
    case S_WRIST_R:  __HAL_TIM_SET_COMPARE(&htim4,  TIM_CHANNEL_3,
(uint32_t)pulseUs); break;
    case S_WRIST_P:  __HAL_TIM_SET_COMPARE(&htim12, TIM_CHANNEL_2,
(uint32_t)pulseUs); break;
    case S_GRIPPER:  __HAL_TIM_SET_COMPARE(&htim2,  TIM_CHANNEL_3,
(uint32_t)pulseUs); break;
    default: break;
  }
}

void Servo_Step(uint8_t id, int8_t direction)
{
  Servo_WritePulse(id, servoPos[id] + (direction * SERVO_STEP_SIZE));
}

void Servo_Home(void)
{
  Servo_WritePulse(S_BASE,     SERVO_MID_US);
  Servo_WritePulse(S_SHOULDER, SERVO_MID_US);
  Servo_WritePulse(S_ELBOW,    SERVO_MID_US);
  Servo_WritePulse(S_WRIST_R,  SERVO_MID_US);
```

```
  Servo_WritePulse(S_WRIST_P,  SERVO_MID_US);
  Servo_WritePulse(S_GRIPPER,  SERVO_MID_US);
}

/**
 * @brief Play back saved positions.
 *        Byte 14 = stop, byte 16 = pause (byte 15 resumes).
 */
void Run_Sequence(void)
{
  if (stepCount < 2)
  {
    UART_Print("Need at least 2 saved positions.\r\n");
    return;
  }

  UART_Print("Running sequence. Send 14 to stop.\r\n");

  uint8_t rx      = 0;
  uint8_t running = 1;

  while (running)
  {
    for (int i = 0; i < stepCount - 1 && running; i++)
    {
      if (HAL_UART_Receive(&huart7, &rx, 1, 1) == HAL_OK)
      {
        if (rx == 14) { running = 0; break; }
        if (rx == 16)
        {
          UART_Print("Paused. Send 15 to resume.\r\n");
          while (rx != 15 && rx != 14)
            HAL_UART_Receive(&huart7, &rx, 1, 100);
          if (rx == 14) { running = 0; break; }
          UART_Print("Resumed.\r\n");
        }
      }

      for (uint8_t s = 0; s < NUM_SERVOS && running; s++)
      {
        int32_t from = savedSteps[i][s];
        int32_t to   = savedSteps[i + 1][s];
        if (from == to) continue;
```

```
      int8_t dir = (to > from) ? 1 : -1;
      for (int32_t pos = from; pos != to; pos += dir)
      {
        Servo_WritePulse(s, pos);
        HAL_Delay(speedDelay);
        if (HAL_UART_Receive(&huart7, &rx, 1, 1) == HAL_OK && rx == 14)
        { running = 0; break; }
      }
    }
  }
 }

 UART_Print("Sequence stopped.\r\n");
}

/* ============================================================
 *  SHARED HELPERS
 * ============================================================ */
void UART_Print(const char *msg)
{
 HAL_UART_Transmit(&huart7, (uint8_t *)msg, strlen(msg), 100);
}

void Print_Mode(void)
{
 if (currentMode == MODE_ROBOT)
   UART_Print("-- ROBOT MODE -- (F/B/L/R/S  |  'M' to switch)\r\n");
 else
   UART_Print("-- ARM MODE --   (1-12=move 13=save 14=reset 15=run H=home  |  'M' to
switch)\r\n");
}

/**
 * @brief System Clock Configuration
 * @retval None
 */
void SystemClock_Config(void)
{
 RCC_OscInitTypeDef RCC_OscInitStruct = {0};
 RCC_ClkInitTypeDef RCC_ClkInitStruct = {0};

 /** Configure the main internal regulator output voltage
 */
 __HAL_RCC_PWR_CLK_ENABLE();
```

```
  __HAL_PWR_VOLTAGESCALING_CONFIG(PWR_REGULATOR_VOLTAGE_SCALE3);

  /** Initializes the RCC Oscillators according to the specified parameters
  * in the RCC_OscInitTypeDef structure.
  */
  RCC_OscInitStruct.OscillatorType = RCC_OSCILLATORTYPE_HSI;
  RCC_OscInitStruct.HSIState = RCC_HSI_ON;
  RCC_OscInitStruct.HSICalibrationValue = RCC_HSICALIBRATION_DEFAULT;
  RCC_OscInitStruct.PLL.PLLState = RCC_PLL_NONE;
  if (HAL_RCC_OscConfig(&RCC_OscInitStruct) != HAL_OK)
  {
    Error_Handler();
  }

  /** Initializes the CPU, AHB and APB buses clocks
  */
  RCC_ClkInitStruct.ClockType = RCC_CLOCKTYPE_HCLK|RCC_CLOCKTYPE_SYSCLK
                              |RCC_CLOCKTYPE_PCLK1|RCC_CLOCKTYPE_PCLK2;
  RCC_ClkInitStruct.SYSCLKSource = RCC_SYSCLKSOURCE_HSI;
  RCC_ClkInitStruct.AHBCLKDivider = RCC_SYSCLK_DIV1;
  RCC_ClkInitStruct.APB1CLKDivider = RCC_HCLK_DIV1;
  RCC_ClkInitStruct.APB2CLKDivider = RCC_HCLK_DIV1;

  if (HAL_RCC_ClockConfig(&RCC_ClkInitStruct, FLASH_LATENCY_0) != HAL_OK)
  {
    Error_Handler();
  }
}

/**
  * @brief TIM1 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM1_Init(void)
{

  /* USER CODE BEGIN TIM1_Init 0 */

  /* USER CODE END TIM1_Init 0 */

  TIM_ClockConfigTypeDef sClockSourceConfig = {0};
  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};
```

```
TIM_BreakDeadTimeConfigTypeDef sBreakDeadTimeConfig = {0};

/* USER CODE BEGIN TIM1_Init 1 */

/* USER CODE END TIM1_Init 1 */
htim1.Instance = TIM1;
htim1.Init.Prescaler = 179;
htim1.Init.CounterMode = TIM_COUNTERMODE_UP;
htim1.Init.Period = 999;
htim1.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim1.Init.RepetitionCounter = 0;
htim1.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_ENABLE;
if (HAL_TIM_Base_Init(&htim1) != HAL_OK)
{
  Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim1, &sClockSourceConfig) != HAL_OK)
{
  Error_Handler();
}
if (HAL_TIM_PWM_Init(&htim1) != HAL_OK)
{
  Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim1, &sMasterConfig) != HAL_OK)
{
  Error_Handler();
}
sConfigOC.OCMode = TIM_OCMODE_PWM1;
sConfigOC.Pulse = 0;
sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCNPolarity = TIM_OCNPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
sConfigOC.OCIdleState = TIM_OCIDLESTATE_RESET;
sConfigOC.OCNIdleState = TIM_OCNIDLESTATE_RESET;
if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
{
  Error_Handler();
}
if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
{
```

```
    Error_Handler();
  }
  if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_3) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_4) != HAL_OK)
  {
    Error_Handler();
  }
  sBreakDeadTimeConfig.OffStateRunMode = TIM_OSSR_DISABLE;
  sBreakDeadTimeConfig.OffStateIDLEMode = TIM_OSSI_DISABLE;
  sBreakDeadTimeConfig.LockLevel = TIM_LOCKLEVEL_OFF;
  sBreakDeadTimeConfig.DeadTime = 0;
  sBreakDeadTimeConfig.BreakState = TIM_BREAK_DISABLE;
  sBreakDeadTimeConfig.BreakPolarity = TIM_BREAKPOLARITY_HIGH;
  sBreakDeadTimeConfig.AutomaticOutput = TIM_AUTOMATICOUTPUT_DISABLE;
  if (HAL_TIMEx_ConfigBreakDeadTime(&htim1, &sBreakDeadTimeConfig) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM1_Init 2 */

  /* USER CODE END TIM1_Init 2 */
  HAL_TIM_MspPostInit(&htim1);

}

/**
  * @brief TIM2 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM2_Init(void)
{

  /* USER CODE BEGIN TIM2_Init 0 */

  /* USER CODE END TIM2_Init 0 */

  TIM_MasterConfigTypeDef sMasterConfig = {0};
  TIM_OC_InitTypeDef sConfigOC = {0};

  /* USER CODE BEGIN TIM2_Init 1 */
```

```
  /* USER CODE END TIM2_Init 1 */
  htim2.Instance = TIM2;
  htim2.Init.Prescaler = 15;
  htim2.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim2.Init.Period = 19999;
  htim2.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim2.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_PWM_Init(&htim2) != HAL_OK)
  {
    Error_Handler();
  }
  sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
  sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
  if (HAL_TIMEx_MasterConfigSynchronization(&htim2, &sMasterConfig) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
  sConfigOC.Pulse = 0;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_PWM_ConfigChannel(&htim2, &sConfigOC, TIM_CHANNEL_3) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM2_Init 2 */

  /* USER CODE END TIM2_Init 2 */
  HAL_TIM_MspPostInit(&htim2);

}

/**
  * @brief TIM4 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM4_Init(void)
{
```

```
/* USER CODE BEGIN TIM4_Init 0 */

/* USER CODE END TIM4_Init 0 */

TIM_ClockConfigTypeDef sClockSourceConfig = {0};
TIM_MasterConfigTypeDef sMasterConfig = {0};
TIM_OC_InitTypeDef sConfigOC = {0};

/* USER CODE BEGIN TIM4_Init 1 */

/* USER CODE END TIM4_Init 1 */
htim4.Instance = TIM4;
htim4.Init.Prescaler = 15;
htim4.Init.CounterMode = TIM_COUNTERMODE_UP;
htim4.Init.Period = 19999;
htim4.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
htim4.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
if (HAL_TIM_Base_Init(&htim4) != HAL_OK)
{
  Error_Handler();
}
sClockSourceConfig.ClockSource = TIM_CLOCKSOURCE_INTERNAL;
if (HAL_TIM_ConfigClockSource(&htim4, &sClockSourceConfig) != HAL_OK)
{
  Error_Handler();
}
if (HAL_TIM_PWM_Init(&htim4) != HAL_OK)
{
  Error_Handler();
}
sMasterConfig.MasterOutputTrigger = TIM_TRGO_RESET;
sMasterConfig.MasterSlaveMode = TIM_MASTERSLAVEMODE_DISABLE;
if (HAL_TIMEx_MasterConfigSynchronization(&htim4, &sMasterConfig) != HAL_OK)
{
  Error_Handler();
}
sConfigOC.OCMode = TIM_OCMODE_PWM1;
sConfigOC.Pulse = 0;
sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
if (HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_1) != HAL_OK)
{
  Error_Handler();
```

```
  }
  if (HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  if (HAL_TIM_PWM_ConfigChannel(&htim4, &sConfigOC, TIM_CHANNEL_3) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM4_Init 2 */

  /* USER CODE END TIM4_Init 2 */
  HAL_TIM_MspPostInit(&htim4);

}

/**
  * @brief TIM12 Initialization Function
  * @param None
  * @retval None
  */
static void MX_TIM12_Init(void)
{

  /* USER CODE BEGIN TIM12_Init 0 */

  /* USER CODE END TIM12_Init 0 */

  TIM_OC_InitTypeDef sConfigOC = {0};

  /* USER CODE BEGIN TIM12_Init 1 */

  /* USER CODE END TIM12_Init 1 */
  htim12.Instance = TIM12;
  htim12.Init.Prescaler = 15;
  htim12.Init.CounterMode = TIM_COUNTERMODE_UP;
  htim12.Init.Period = 19999;
  htim12.Init.ClockDivision = TIM_CLOCKDIVISION_DIV1;
  htim12.Init.AutoReloadPreload = TIM_AUTORELOAD_PRELOAD_DISABLE;
  if (HAL_TIM_PWM_Init(&htim12) != HAL_OK)
  {
    Error_Handler();
  }
  sConfigOC.OCMode = TIM_OCMODE_PWM1;
```

```
  sConfigOC.Pulse = 0;
  sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
  sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;
  if (HAL_TIM_PWM_ConfigChannel(&htim12, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN TIM12_Init 2 */

  /* USER CODE END TIM12_Init 2 */
  HAL_TIM_MspPostInit(&htim12);

}

/**
  * @brief UART7 Initialization Function
  * @param None
  * @retval None
  */
static void MX_UART7_Init(void)
{

  /* USER CODE BEGIN UART7_Init 0 */

  /* USER CODE END UART7_Init 0 */

  /* USER CODE BEGIN UART7_Init 1 */

  /* USER CODE END UART7_Init 1 */
  huart7.Instance = UART7;
  huart7.Init.BaudRate = 115200;
  huart7.Init.WordLength = UART_WORDLENGTH_8B;
  huart7.Init.StopBits = UART_STOPBITS_1;
  huart7.Init.Parity = UART_PARITY_NONE;
  huart7.Init.Mode = UART_MODE_TX_RX;
  huart7.Init.HwFlowCtl = UART_HWCONTROL_NONE;
  huart7.Init.OverSampling = UART_OVERSAMPLING_16;
  if (HAL_UART_Init(&huart7) != HAL_OK)
  {
    Error_Handler();
  }
  /* USER CODE BEGIN UART7_Init 2 */

  /* USER CODE END UART7_Init 2 */
```

```
}

/**
  * @brief GPIO Initialization Function
  * @param None
  * @retval None
  */
static void MX_GPIO_Init(void)
{
  GPIO_InitTypeDef GPIO_InitStruct = {0};
  /* USER CODE BEGIN MX_GPIO_Init_1 */

  /* USER CODE END MX_GPIO_Init_1 */

  /* GPIO Ports Clock Enable */
  __HAL_RCC_GPIOC_CLK_ENABLE();
  __HAL_RCC_GPIOA_CLK_ENABLE();
  __HAL_RCC_GPIOE_CLK_ENABLE();
  __HAL_RCC_GPIOB_CLK_ENABLE();
  __HAL_RCC_GPIOD_CLK_ENABLE();

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOC, GPIO_PIN_0|GPIO_PIN_4|GPIO_PIN_5, GPIO_PIN_RESET);

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);

  /*Configure GPIO pin Output Level */
  HAL_GPIO_WritePin(GPIOD, GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                    |GPIO_PIN_4|GPIO_PIN_5, GPIO_PIN_RESET);

  /*Configure GPIO pins : PC0 PC4 PC5 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_4|GPIO_PIN_5;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOC, &GPIO_InitStruct);

  /*Configure GPIO pin : PA5 */
  GPIO_InitStruct.Pin = GPIO_PIN_5;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
```

```
  HAL_GPIO_Init(GPIOA, &GPIO_InitStruct);

  /*Configure GPIO pins : PD0 PD1 PD2 PD3
                         PD4 PD5 */
  GPIO_InitStruct.Pin = GPIO_PIN_0|GPIO_PIN_1|GPIO_PIN_2|GPIO_PIN_3
                        |GPIO_PIN_4|GPIO_PIN_5;
  GPIO_InitStruct.Mode = GPIO_MODE_OUTPUT_PP;
  GPIO_InitStruct.Pull = GPIO_NOPULL;
  GPIO_InitStruct.Speed = GPIO_SPEED_FREQ_LOW;
  HAL_GPIO_Init(GPIOD, &GPIO_InitStruct);

  /* USER CODE BEGIN MX_GPIO_Init_2 */

  /* USER CODE END MX_GPIO_Init_2 */
}

/* USER CODE BEGIN 4 */

/* USER CODE END 4 */

/**
  * @brief  This function is executed in case of error occurrence.
  * @retval None
  */
void Error_Handler(void)
{
  /* USER CODE BEGIN Error_Handler_Debug */
  /* User can add his own implementation to report the HAL error return state */
  __disable_irq();
  while (1)
  {
  }
  /* USER CODE END Error_Handler_Debug */
}
#ifdef USE_FULL_ASSERT
/**
  * @brief  Reports the name of the source file and the source line number
  *         where the assert_param error has occurred.
  * @param  file: pointer to the source file name
  * @param  line: assert_param error line source number
  * @retval None
  */
void assert_failed(uint8_t *file, uint32_t line)
{
```

```
  /* USER CODE BEGIN 6 */
  /* User can add his own implementation to report the file name and line number,
     ex: printf("Wrong parameters value: file %s on line %d\r\n", file, line) */
  /* USER CODE END 6 */
}
#endif /* USE_FULL_ASSERT */
```